

Pose & Resample Reference

Every command to install, build, find a projection center-of-rotation pose, resample against it, and validate the result, across the three interchangeable backends of the `projection-center` command and the course-required pybind11 Python interface.

● `opencl` · PyOpenCL, GPU

● `cpu` · NumPy, no GPU

● `cpp` · `projection-center-cpp/` binaries

● `pybind` · Python interface

01 Setup

Run once, from the repo root (`Topic_3/`). The second block is Linux-only, and only needed for the `cpp` backend.

Install the Python package

Editable install; registers the `projection-center` command.

```
cd projection-center-python-opencl && python3 -m pip install -e . && cd ..
```

Build the C++/OpenCL binaries (Linux only)

Configures, then compiles, via meson and ninja. Needs (Ubuntu/Debian) `ocl-icd-opencl-dev` `libhdf5-dev` `libboost-program-options-dev` `libboost-json-dev` (on Debian 12, which has no `libboost-json-dev`: `libboost-program-options1.81-dev` `libboost-json1.81-dev` instead). The OpenCL headers are bundled in `projection-center-cpp/third_party/opencl/`.

```
cd projection-center-cpp && meson setup builddir && meson compile -C builddir && cd ..
```

List visible OpenCL devices

Queries every OpenCL platform and prints what it finds.

```
projection-center devices
```

Python interface (pybind11): check the requirements

pybind

Needs `torch` (brings `torch.utils.cpp_extension` and the pybind11 headers), `numpy`, `h5py`, plus `g++` and `ninja`. OpenCL loader: `sudo apt install ocl-icd-opencl-dev` (the headers are bundled). No build step: the first import compiles the module (~40 s) and caches it.

```
python3 -c "import torch, numpy, h5py" && which g++ ninja
```

02 Datasets

The two course datasets live under `data/` (gitignored, populate locally). The commands below spell out every parameter explicitly, nothing left to a default.

`data/proj_shepplogan128.hdf5` · Shepp–Logan phantom, 128px

projections	500
detector (H × W)	128 × 128
SDD / SOD	1085.6 / 416.5
pixel size	1.731

```
projection-center pipeline --backend opengl --data data/proj_shepplogan128.hdf5 --xshift 40.0 --alpha 10.0 --beta 10.0 --xshift-step 1.0 --alpha-step 1.0 --beta-step 1.0 --downsample 1 --batch-size 16 --output-pose runs/shepp128_run/pose.json --output-data runs/shepp128_run/resampled.hdf5
```

`data/proj_shepplogan512.hdf5` · Shepp–Logan phantom, 512px

projections	1000
detector (H × W)	800 × 500
SDD / SOD	1.2 / 0.2
pixel size	0.00117

```
projection-center pipeline --backend opengl --data data/proj_shepplogan512.hdf5 --xshift 40.0 --alpha 10.0 --beta 10.0 --xshift-step 1.0 --alpha-step 1.0 --beta-step 1.0 --downsample 1 --batch-size 16 --output-pose runs/shepp512_run/pose.json --output-data runs/shepp512_run/resampled.hdf5
```

▲ **Shepp–Logan caveat:** on both phantoms part of the found pose sits on the edge of the search range (128px: xshift = 40 mm; 512px: beta = 10°), identically on every backend. That's a property of the phantom's own symmetry, not a bug (see Limitations). Trust alpha/beta on 128px; treat edge values as weakly determined.

Swap `--backend` for `cpu` / `cpp` exactly as in the command matrix below. Every flag here works identically on either dataset, on any backend.

03 The command matrix

Every subcommand runs on every backend, nine combinations in total. Swap `--backend`, everything else stays the same. Run from the repo root.

search

Find the pose

opengl

Grid-searches the pose on the GPU with the package's own PyOpenCL kernel.

```
projection-center search --backend opengl --data data/proj_shepplogan128.hdf5 --  
output-pose runs/my_run/pose.json
```

Find the pose

cpu

Same grid search, vectorized in NumPy, no GPU involved.

```
projection-center search --backend cpu --data data/proj_shepplogan128.hdf5 --output-  
pose runs/my_run/pose.json
```

Find the pose

cpp

Runs the compiled `projection-center-cpp/build/forward_search` binary and reads its result back.

```
projection-center search --backend cpp --data data/proj_shepplogan128.hdf5 --output-  
pose runs/my_run/pose.json --cpp-mode buffer
```

resample

Apply a pose

opengl

Ray-casts every output pixel back into the original image, on the GPU.

```
projection-center resample --backend opengl --data data/proj_shepplogan128.hdf5 --  
pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

Apply a pose

cpu

Same ray-casting, vectorized in NumPy.

```
projection-center resample --backend cpu --data data/proj_shepplogan128.hdf5 --pose  
runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

Apply a pose

cpp

Runs the compiled `forward_search_resample` binary.

```
projection-center resample --backend cpp --data data/proj_shepplogan128.hdf5 --pose  
runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5 --cpp-mode buffer
```

pipeline

Search, then resample

opengl

Runs the two commands above back to back, same backend for both.

```
projection-center pipeline --backend opengl --data data/proj_shepplogan128.hdf5 --  
output-pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

Search, then resample

cpu

Same, entirely on CPU.

```
projection-center pipeline --backend cpu --data data/proj_shepplogan128.hdf5 --  
output-pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5
```

Search, then resample

cpp

Same, via the single-process `projection_center_pipeline_cpp` binary.

```
projection-center pipeline --backend cpp --data data/proj_shepplogan128.hdf5 --  
output-pose runs/my_run/pose.json --output-data runs/my_run/resampled.hdf5 --cpp-  
mode buffer
```

① A fourth choice, `--backend hybrid`, runs the `cpp` search with the `opencl` resample kernel. Same flags as above.

✓ **Confirmed working, all nine plus hybrid**, on the 128px phantom: the same winning pose on every backend (xshift = 40 mm, alpha = -3° , beta = 4° , MSE $6.594e-4$), matching the bug-fixed reference. Resampled pixels agree to a mean difference of at most 0.0015 on a data range of $\sim 5 \cdot 10^4$. Pipeline time: `cpp` 1.4 s, `opencl` 1.9 s, `cpu` 26 s.

04 Python interface (pybind11)

The interface the course requires: the OpenCL code compiled into a Python module with `torch.utils.cpp_extension` + `pybind11`. Python reads the data, passes it in, and gets the results back. Laid out like the course's `pybindextension.zip`: `projection-center-cpp/backend.py` next to a `src/` folder. Full manual: `repo_docs/pybind-interface.md`.

Go to the interface's folder

pybind

All `run_pybind.py` commands below run from here.

```
cd projection-center-cpp
```

- ① From the repo root instead: `python3 projection-center-cpp/run_pybind.py --data data/proj_shepplogan128.hdf5`. Running `python3 run_pybind.py` from the repo root fails with `can't open file`, because the script is one folder down. The `No CUDA runtime is found` warning is harmless: this project uses OpenCL.

Full flow: read HDF5 → search → resample → write HDF5

pybind

Writes the pose to `runs/real_cb_pose.h5` and the corrected projections to `runs/projs_resample.h5`, and prints the stage log, the pose, `search total`, `resample total`, a timing line (`read`, `compute`, `write`, `I/O`, `compute+I/O`) and `total run`, plus the one-off import time at the start. The first run takes longer while it compiles the module.

```
python3 run_pybind.py --data ../data/proj_shepplogan128.hdf5
python3 run_pybind.py --data ../data/proj_shepplogan512.hdf5
```

Pick the implementation

pybind

`cpp` (default): our OpenCL kernels · `opengl`: the package's OpenCL kernels, run from C++ · `cpu`: the package's NumPy backend, ported to C++.

```
python3 run_pybind.py --data ../data/proj_shepplogan128.hdf5 --backend opengl
python3 run_pybind.py --data ../data/proj_shepplogan128.hdf5 --backend cpu
```

Search only / keep separate outputs

pybind

Every run overwrites the same two output files unless given its own names.

```
python3 run_pybind.py --data ../data/proj_shepplogan128.hdf5 --skip-resample
python3 run_pybind.py --data ../data/proj_shepplogan512.hdf5 --output runs/pose_512.h5 --resample-
output runs/resampled_512.h5
```

From your own Python code

pybind

Run from `projection-center-cpp/`. Search ranges in mm/degrees; the pose comes back in meters/radians.

```
import h5py
from backend import _backend

with h5py.File("../data/proj_shepplogan128.hdf5", "r") as f:
    projs = f["Projection"][()]
    SDD, SOD, px = f["SDD"][()], f["SOD"][()], f["pixelSize"][()]
H, W = projs.shape[1:]

pose = _backend.search(projs, SDD, SOD, px, W, H, xshift=40.0, alpha=10.0, beta=10.0,
                      xshift_step=1.0, alpha_step=1.0, beta_step=1.0)  #
backend="cpp"|"opengl"|"cpu"
out = _backend.resample(projs, SDD, SOD, px, xshift=pose["xshift"], alpha=pose["alpha"],
                      beta=pose["beta"], center_x=pose["center_x"], center_y=pose["center_y"])
corrected = out["projections"]  # numpy array (num_projs, H, W)
```

Tests

pybind

From `projection-center-cpp/`. Each prints ... PASS.

```
python3 tests/test_backend_smoke.py
python3 tests/test_cli_vs_backend.py
python3 tests/test_resample_cli_vs_backend.py
python3 tests/test_ported_backends.py
```

- ✓ **Confirmed working:** every command above, from both folders, plus the four tests (all PASS). 128px: 40 mm / -3° / 4° on cpp, opencl and cpu; 512px: 35 mm / -1° / 10° , the same poses as the `projection-center` backends.

05 Optional flags

`projection-center` : same meaning on every backend unless noted. Defaults shown.

FLAG	DEFAULT	APPLIES TO
<code>--xshift / --alpha / --beta</code>	40.0 / 10.0 / 10.0	search, pipeline · $\pm$ range, mm / deg
<code>--xshift-step / --alpha-step / --beta-step</code>	1.0 / 1.0 / 1.0	search, pipeline
<code>--sample-count / --sample-angle-range</code>	1000 / 30.0	search, pipeline · opencl and cpu only
<code>--downsample</code>	1	resample, pipeline
<code>--batch-size</code>	16	resample, pipeline · GPU batching only
<code>--platform-index / --device-index</code>	first device	opencl only
<code>--cpp-mode</code>	buffer	cpp only · image buffer

cpp does not support non-default `--sample-count / --sample-angle-range` : `projection-center-cpp/` hardcodes them (1000 / 30°).

`run_pybind.py` (pybind interface), run from `projection-center-cpp/` :

FLAG	DEFAULT	MEANING
<code>--data</code>	required	input HDF5 file
<code>--backend</code>	cpp	cpp opencl cpu
<code>--xshift / --alpha / --beta</code>	40 / 10 / 10	$\pm$ search range, mm / deg
<code>--xshift-step / --alpha-step / --beta-step</code>	1 / 1 / 1	search step, mm / deg
<code>--mode</code>	buffer	cpp backend only · image buffer
<code>--downsample</code>	1	detector downsample factor for resample
<code>--output / --resample-output</code>	runs/real_cb_pose.h5 / runs/projs_resample.h5	output files
<code>--skip-resample</code>	off	only run the search

06 Untouched Python reference

Serial, slow, must stay unmodified. Everything above validates against this.

Forward search · writes `./real_cb_pose.json`

Plain Python loop over every candidate pose, one at a time. On the 128px phantom it stops with `UnboundLocalError` (the known reference bug, see Limitations); use the pipeline command below there.

```
python3 reference/Topic_3_forwardsearching.py --data data/proj_shepplogan128.hdf5
```

Resampling · reads `./real_cb_pose.json`

Plain Python loop over every output pixel. No `--pose` flag: always reads the file the step above wrote. Writes to a hardcoded lab-network path, so it only completes on that network.

```
python3 reference/Topic_3_resampling.py --data data/proj_shepplogan128.hdf5
```

Recommended · both steps, anywhere · ~6 minutes on 128px

Runs the reference's own search and resample functions in-process, with the known bug patched in memory (like `--fix-ref`) and the hardcoded write path bypassed. The files on disk are never touched. Works on the 128px phantom.

```
python3 reference/run_reference_pipeline.py --data data/proj_shepplogan128.hdf5 --output-pose runs/reference/pose.json --output-data runs/reference/resampled.hdf5
```

07 Validating a pose against the reference

Three ways to run it, in increasing rigor. `--gpu` takes a pose from any backend (`.json` or `.h5`). With `--run-ref`, the new reference pose is saved to a new `runs/ref_cb_pose_<timestamp>.json` (the path is printed), so earlier results are never overwritten; pass `--ref` to pick the file yourself.

Against an existing reference pose file

Just diffs two already-computed poses; runs nothing new.

```
python3 validate_forward_search.py --gpu runs/my_run/pose.json --ref runs/reference/pose.json
```

Run the raw reference now (~5.5 min on 128px)

Runs the untouched reference script itself, bug included, as the baseline. On the 128px phantom it stops with the same `UnboundLocalError`; use the recommended form below.

```
python3 validate_forward_search.py --gpu runs/my_run/pose.json --run-ref --data  
data/proj_shepplogan128.hdf5
```

Recommended · bug-fixed reference, in-process only

Same run, with the reference's bug patched in memory only, file untouched.

```
python3 validate_forward_search.py --gpu runs/my_run/pose.json --run-ref --fix-ref --data  
data/proj_shepplogan128.hdf5
```

✓ **Confirmed:** the first and third commands report `OVERALL: PASS` on the 128px phantom.

❗ **Why `--fix-ref`:** the raw reference has a known bug (see Limitations) that can shift its own optimum. The corrected run patches it in memory only; the reference file is never touched.

08 Benchmarking all backends

Two scripts, both run from the repo root.

Every backend, 10× each · `benchmark_pybind.py`

Times pybind-cpp/opencl/cpu and the projection-center cpp/opencl/cpu backends, each rep a fresh process doing read → search → resample → write. Prints mean ± stdev plus per-stage times for pybind; raw results appended to `runs/pybind_10x_results.jsonl`.

```
python3 benchmark_pybind.py
python3 benchmark_pybind.py --reps 10 --datasets 128 --backends pybind-cpp pybind-opencl pybind-cpu
```

Against the raw reference · `benchmark_backends.py`

Runs the raw reference, then opencl/cpu/cpp once each, and prints one timing + pose table. Each run saves its outputs in its own new folder, `runs/benchmark_all/<timestamp>/<name>/`, so earlier results are never overwritten; `--output-dir` picks the folder yourself. The reference resample is very slow; skip it or the whole reference.

```
python3 benchmark_backends.py --data data/proj_shepplogan128.hdf5 --skip-reference-resample
python3 benchmark_backends.py --data data/proj_shepplogan128.hdf5 --skip-reference
```

- ❗ Don't run two benchmarks at once, or anything else GPU-heavy: they compete for the same GPU and both sets of timings come out wrong. `import torch` costs ~5 s once per process for the pybind runs, listed separately as the `import` stage.

09 Comparing backends yourself

Differs two already-produced resampled HDF5 files pixel by pixel, e.g.

`runs/benchmark_all/<timestamp>/opencl/resampled.hdf5` against `../cpu/resampled.hdf5` from the same benchmark run.

Quick · first 5 projections

Loads just 5 projections from each file and reports the pixel-wise difference.

```
python3 -c "  
import h5py, numpy as np  
a = h5py.File('a/resampled.hdf5')['Projection'][:5]  
b = h5py.File('b/resampled.hdf5')['Projection'][:5]  
d = np.abs(a.astype(float)-b.astype(float))  
print(d.max(), d.mean())"
```

Full · every projection

Same idea, over the whole dataset instead of a sample.

```
python3 -c "  
import h5py, numpy as np  
a = h5py.File('a/resampled.hdf5')['Projection'][:]  
b = h5py.File('b/resampled.hdf5')['Projection'][:]  
d = np.abs(a.astype(float)-b.astype(float))  
print(d.max(), d.mean(), np.median(d))"
```

10 Screenshots

Output is raw float32 HDF5. Pull a slice, save as PNG. Both commands use the same projection index, so the two PNGs are directly comparable. Need `pillow`.

Original (before correction) → PNG

```
python3 -c "  
import h5py, numpy as np  
from PIL import Image  
p = h5py.File('data/proj_shepplogan128.hdf5')['Projection'][0]  
n = (p - p.min()) / (p.max() - p.min())  
Image.fromarray((n*255).astype(np.uint8)).save('projection_0_original.png')"
```

Resampled (corrected) → PNG

```
python3 -c "  
import h5py, numpy as np  
from PIL import Image  
p = h5py.File('runs/my_run/resampled.hdf5')['Projection'][0]  
n = (p - p.min()) / (p.max() - p.min())  
Image.fromarray((n*255).astype(np.uint8)).save('projection_0.png')"
```

11 Limitations

- ▲ `cpp` is verified on Linux. On Windows it compiles via MinGW-w64/MSYS2, but running it on real data isn't verified yet (see [repo_docs/windows-support.pdf](#)).
- ▲ `--cpp-mode image / mode="image"` crashes on the dev machine's Intel iGPU driver. `buffer` is what's validated.
- ▲ The reference has a known bug: a ray that misses the detector can silently reuse the previous ray's value, or, on the 128px phantom, crash with `UnboundLocalError` . Use `--fix-ref` or `reference/run_reference_pipeline.py` , which patch it in memory only.
- ▲ `Topic_3_resampling.py` writes to a hardcoded lab-network path, so it fails off that network. Use `reference/run_reference_pipeline.py` or `projection-center resample` instead.
- ▲ On the Shepp–Logan phantoms part of the found pose is weakly determined and lands on the search-range edge (128px: xshift; 512px: beta), identically on every backend: a property of the phantom, not a bug.
- ▲ `pybind` runs pay ~5 s for `import torch` , once per Python process: a fixed cost of the required `torch.utils.cpp_extension` interface, not of the computation.